

锂离子电池快充策略对寿命衰减的影响建模与优化

摘 要

提高充电倍率可缩短补能时间,但过高电流会加速容量衰减。本文基于 MIT-Stanford 公开的 49 块 A123 18650 型磷酸铁锂电池循环老化数据,围绕两阶段快充策略 $C_1-Q_1-C_2$, 研究快充参数对寿命衰减的影响,并讨论寿命预测与策略优化问题。

针对问题一,本文对循环数据进行逐电池、逐字段的 IQR 异常检测与前后邻域均值填补,共替换 559 个异常单元格;提取 9 种快充策略,并以第 200 次循环的 SOH 作为观测窗口内的寿命代理指标(数据尚未覆盖 $\text{SOH} = 80\%$ 的真实寿命终止)。在 40 块满 200 圈电池中,SOH 均值为 0.9914,按 25%/75% 分位数划分得到长寿命 10 块、中等寿命 20 块、短寿命 10 块。策略层面,寿命最长为 $3.6C(80\%) - 3.6C$,寿命最短为 $3.7C(31\%) - 5.9C$ (NEWSTRUCTURE)。同名参数但带/不带 NEWSTRUCTURE 标签的策略寿命差异明显,提示批次或实验结构不可忽略。

针对问题二,拟在问题一基础上检验策略间寿命差异的统计显著性,分析 C_1, Q_1, C_2 与寿命代理指标的关系,并讨论不同 SOC 区间高倍率充电的衰减差异及机制局限。(本节定量结果待补充)

针对问题三,拟利用截至第 150 次循环的早期数据,提取健康特征并建立 SOH 衰减预测模型,预报第 151–200 次循环 SOH,并外推至 $\text{SOH} = 80\%$ 的循环寿命;同时比较不同早期窗口长度及策略信息的作用。(本节定量结果待补充)

针对问题四,拟建立充电时间模型与寿命衰减模型,在已有实验策略及其合理邻域内进行充电时间–寿命的多目标权衡,给出推荐策略并讨论外推风险。(本节定量结果待补充)

关键词: 锂离子电池; 快充策略; SOH; 循环寿命; IQR; 寿命预测; 多目标优化

1 问题重述

随着新能源汽车与储能系统对补能效率的要求提高，锂离子电池快充成为缩短充电时间的主要手段。然而较高充电电流会增强极化、加剧热效应并加速副反应，从而加快容量衰减、缩短循环寿命 [1–3]。电池健康状态定义为

$$\text{SOH} = \frac{Q_{\text{current}}}{Q_{\text{initial}}}, \quad (1)$$

本题以 $\text{SOH} = 80\%$ 作为寿命终止 (EOL) 判定阈值。

本题充电过程采用两阶段快充：以倍率 C_1 从 0% SOC 充至切换点 Q_1 ，再以倍率 C_2 从 Q_1 充至 80% SOC，随后以 1C 恒流–恒压 (CC-CV) 补至满电，至电流降至 $C/50$ 结束。各电池放电条件一致，主要差异在充电策略。数据来自 MIT–Stanford 公开循环老化实验 [1, 4]，包含 49 块额定容量 1.1 Ah 的 A123 18650 磷酸铁锂电池。

需要完成的研究任务为：

- 问题 1.** 整理数据，提取策略、充电时间、SOH 变化与循环寿命信息，分析寿命分布并解释典型长/短寿命策略差异。
- 问题 2.** 分析策略参数 C_1, Q_1, C_2 对循环寿命的影响及其统计显著性与可能机制。
- 问题 3.** 基于截至第 150 次循环的数据预测第 151–200 次循环 SOH，并外推 EOL 循环寿命。
- 问题 4.** 兼顾充电时间与寿命衰减，在实验策略及其合理邻域内优化快充策略。

2 问题分析与总体思路

问题一属于探索性数据分析：在尚未达到真实 EOL 的观测窗口内，需要定义可比较的寿命代理指标，并完成清洗、可视化与典型策略提取。问题二强调参数非正交，不宜按完全析因实验解释因果关系。问题三是早期循环预测与外推。问题四是充电时间与衰减率的多目标决策。四问递进关系为：数据整理 → 影响分析 → 寿命预测 → 策略优化。

待完成 / 框架占位

问题二至问题四的模型推导、数值表与最终推荐策略尚未写入仓库，下文相应小节仅给出可直接填充的结构、符号与拟采用方法，待后续结果补入后删除本框并替换为正式论述。

3 符号说明与基本假设

3.1 主要符号

表 1 主要符号说明

符号	含义
SOH	电池健康状态, 见式(1)
C_1, C_2	第一、二阶段充电倍率 (C-rate)
Q_1	阶段切换 SOC (%)
N	循环次数
N_{EOL}	SOH = 80% 对应的循环寿命
SOH_{200}	第 200 次循环的 SOH (寿命代理)
T_{charge}	平均充电时间 (min)

3.2 基本假设

1. 各电池放电协议一致, 寿命差异主要由充电策略及电池个体差异引起。
2. 在 200 次循环窗口内未达到 SOH = 80%, 故问题一以 SOH_{200} 作为寿命代理指标, 并在文中明确其与真实循环寿命的区别。
3. 带 NEWSTRUCTURE 后缀的策略与同名无后缀策略视为不同实验结构/批次标签, 分析时不合并。
4. IQR 筛出的离群点视为统计异常而非已证实的传感器故障, 清洗仅用于本题指定分析。
5. 问题四若扩展连续参数, 取值限制在已有实验数据的凸包或合理邻域内, 避免无约束外推。

4 问题一：数据整理与快充策略对寿命影响的初步分析

4.1 数据来源与结构

本题使用赛题提供的 MIT-Stanford 锂离子电池循环老化数据 [1,4]。battery_summary.csv 记录 49 块电池的策略参数 C_1, Q_1, C_2 、初始容量、平均充电时间、平均内阻、平均温度及问题三测试标记 prediction_test; cycle_train.csv 记录逐循环的容量、SOH、平滑 SOH、充电时间、内阻与平均温度, 共 9350 行。非测试电池记录至第 200 次循环, 9 块测试电池仅至第 150 次循环。输入质量检查表明: battery_id+cycle 无重复; 循环表无缺失; 汇总表有 3 个缺失单元格, 其中 C_1 的结构性缺失予以保留, 不参与循环数据清洗。最大循环数分布为 {150 : 9, 200 : 40}。

4.2 寿命指标定义

题设循环寿命为 SOH 降至 80% 时的循环次数 N_{EOL} 。本数据集在 200 次循环内 SOH_{200} 范围为 0.9497 ~ 1.0002，均值 0.9914，尚未覆盖真实寿命终止。因此问题一采用

$$LifeProxy_i = SOH_i(N = 200) \quad (2)$$

作为观测窗口内的使用寿命代理：该值越高，表示前 200 圈健康保持越好。后文箱线图、分布图与策略标签均基于该代理指标。若需与问题三衔接，可进一步计算衰减率

$$FadeRate_i = \frac{SOH_i(1) - SOH_i(N)}{N - 1}, \quad (3)$$

并在问题二、三中统一使用。[1,5]

4.3 IQR 清洗

对每块电池、每个测量字段 (capacity、SOH、SOH_smooth、chargetime、IR、Tavg) 分别计算下、上四分位数 Q_1, Q_3 及四分位距 $IQR = Q_3 - Q_1$ 。将

$$x \notin [Q_1 - 1.5 IQR, Q_3 + 1.5 IQR]$$

的观测标为异常，并用同一电池、同一字段中距离最近的前后正常值均值替换；若仅有单侧邻域则用该侧值。共替换 559 个单元格，分字段数量为：capacity 36、SOH 36、SOH_smooth 17、IR 142、Tavg 142、chargetime 186。原始 data/ 文件未改写，清洗结果保存为 output/A/cycle_train_cleaned.csv。

4.4 策略提取与 200 圈样本

由汇总表提取 49 块电池的 battery_id-policy 对照，得到 9 种策略（表 2 给出示意，完整表见支撑材料 charge_policy.csv）。寿命统计分析仅纳入最大循环数恰为 200 的 40 块电池，排除 9 块问题三测试电池，以免 150 圈终点与 200 圈终点混比。

表 2 部分电池快充策略提取结果（完整 49 行见支撑材料）

battery_id	policy
1	3_6C-80PER_3_6C
4	80PER_3_6C
7	4_8C_80PER_4_8C
10	5C_67PER_4C_NEWSTRUCTURE
16	3_7C_31PER_5_9C_NEWSTRUCTURE
49	4_8C_80PER_4_8C_NEWSTRUCTURE

4.5 SOH 变化曲线

以 1 号电池为例绘制清洗后 SOH 与平滑 SOH 随循环次数的变化，见图 1。早期循环中 SOH 可略高于 1（容量活化/测量波动），随后缓慢下降。该图用于展示单电池衰减形态，策略间对比见后文分布图与箱线图。

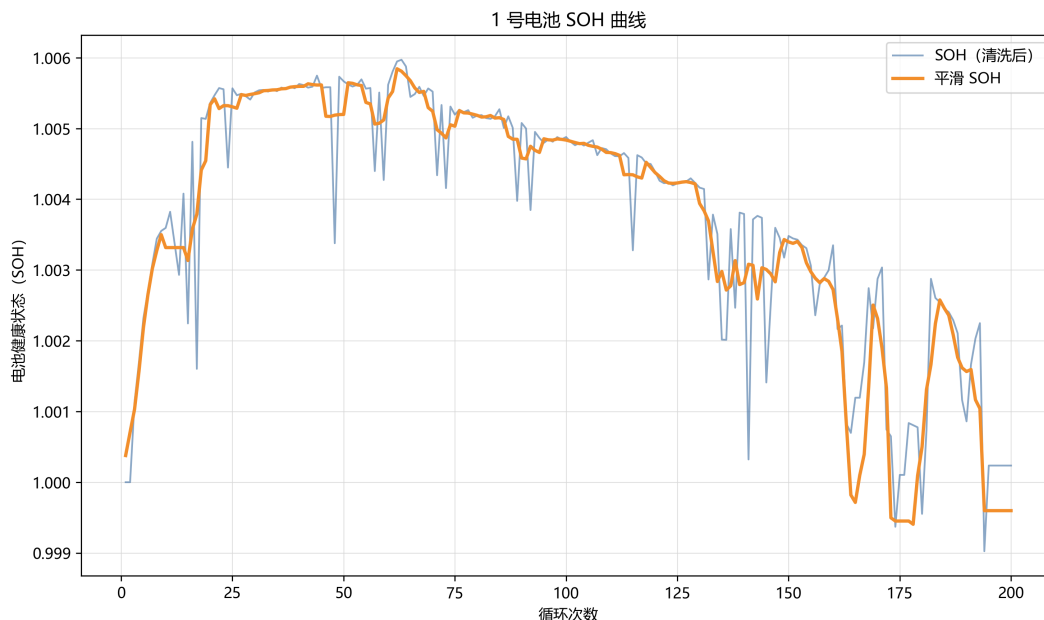


图 1 1 号电池 SOH 曲线（清洗后原始 SOH 与平滑 SOH）

4.6 寿命分布统计分析

对 40 块电池的 SOH_{200} 绘制直方图。取样本 75% 分位数

$$critical_SOH_{high} = 0.996103$$

为长寿命分界（红色虚线），25% 分位数

$$critical_SOH_{low} = 0.993433$$

为短寿命分界（蓝色虚线）。据此分类：长寿命 10 块、中等寿命 20 块、短寿命 10 块。分布见图 2。

SOH_{200} 与平均充电时间的散点图见图 3，按策略着色。可见充电时间较短的策略并不一律对应更低 SOH，时间-寿命关系并非简单单调，为问题四的多目标权衡提供现象基础。

不同策略下 SOH_{200} 的箱线图见图 4，并叠加上述两条分界线。各策略汇总统计见表 3。

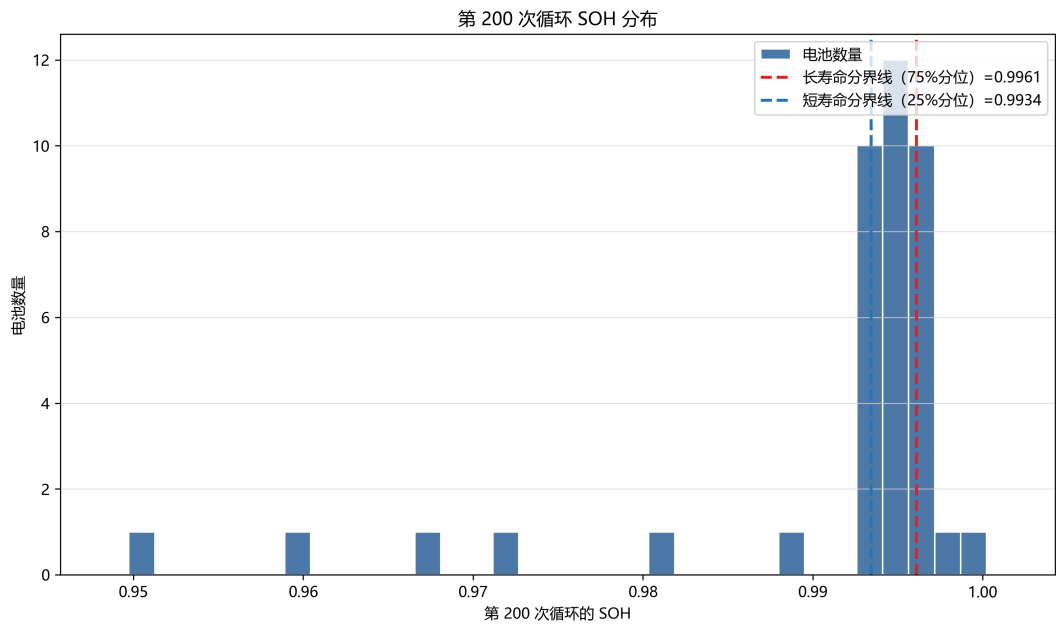


图 2 第 200 次循环 SOH 分布及长/短寿命分位数分界线

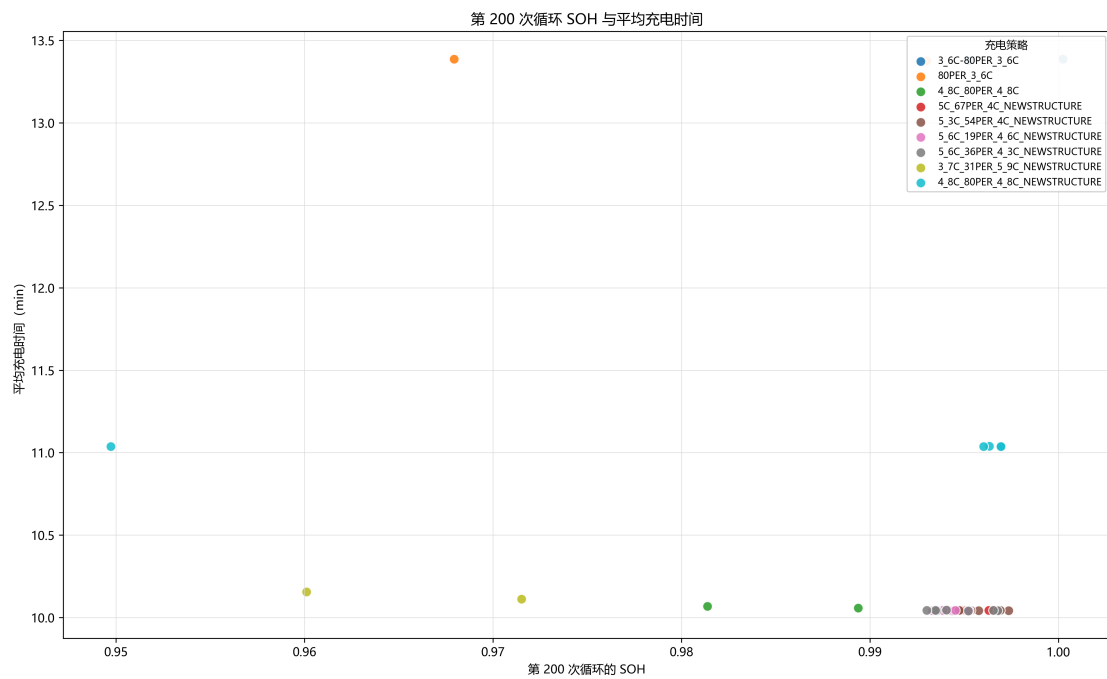


图 3 第 200 次循环 SOH 与平均充电时间散点图（按策略着色）

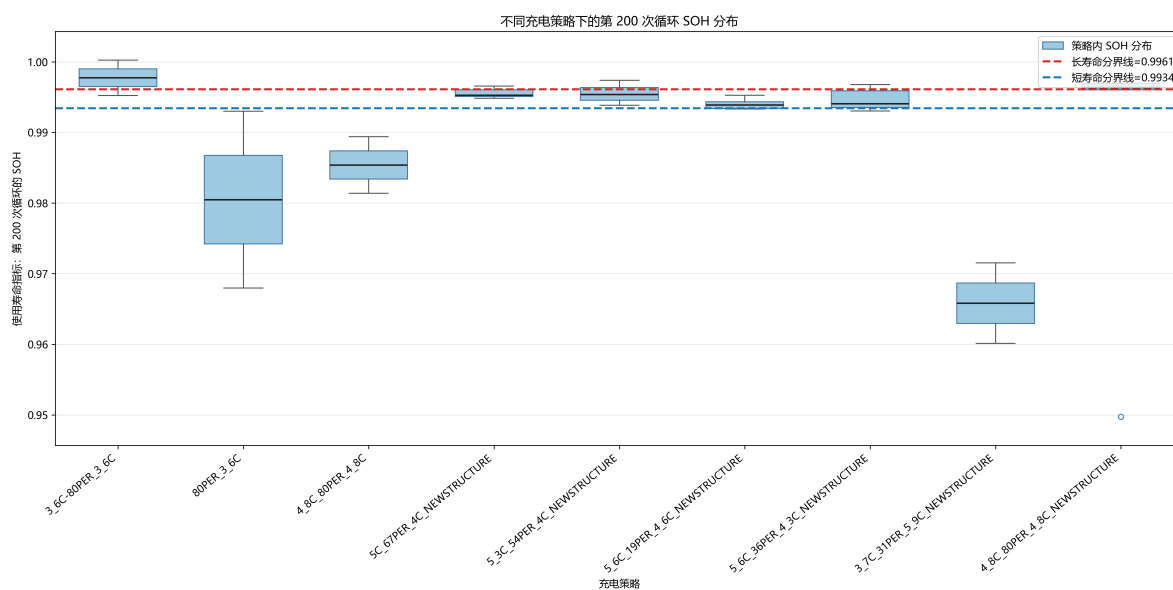


图 4 不同充电策略下第 200 次循环 SOH 箱线图

表 3 各充电策略第 200 次循环 SOH 与充电时间汇总（仅 200 圈样本）

策略	n	SOH 中位数	SOH 范围	充电时间范围/min	标签
3_6C-80PER_3_6C	2	0.9977	0.9952–1.0002	13.378–13.386	长寿命
4_8C_80PER_4_8C_NEWSTRUCTURE	5	0.9963	0.9497–0.9970	11.038–11.038	长寿命
5C_67PER_4C_NEWSTRUCTURE	6	0.9952	0.9948–0.9966	10.042–10.044	中等
5_3C_54PER_4C_NEWSTRUCTURE	7	0.9954	0.9938–0.9974	10.042–10.044	中等
5_6C_36PER_4_3C_NEWSTRUCTURE	7	0.9941	0.9930–0.9968	10.041–10.045	中等
5_6C_19PER_4_6C_NEWSTRUCTURE	7	0.9939	0.9933–0.9953	10.042–10.044	中等
4_8C_80PER_4_8C	2	0.9854	0.9814–0.9894	10.058–10.068	短寿命
80PER_3_6C	2	0.9805	0.9679–0.9930	13.376–13.388	短寿命
3_7C_31PER_5_9C_NEWSTRUCTURE	2	0.9658	0.9601–0.9715	10.111–10.156	短寿命

注：策略类型按策略内 SOH 中位数与总体 25%/75% 分位阈值比较得到。 n 为满 200 圈电池数。

4.7 典型长寿命与短寿命策略

入选规则：以策略内 SOH_{200} 中位数为序；长寿命要求中位数 ≥ 0.996103 ，短寿命要求中位数 ≤ 0.993433 。

寿命最长策略：3_6C-80PER_3_6C，中位数 0.997734，范围 0.995232–1.000235，平均充电时间约 13.38 min ($n = 2$)。

典型长寿命策略：4_8C_80PER_4_8C_NEWSTRUCTURE，中位数 0.996334，但范围宽至 0.949733–0.996955，含一个明显偏低个体，解读时需结合离散程度。

寿命最短策略：3_7C_31PER_5_9C_NEWSTRUCTURE，中位数 0.965818，范围 0.960114–0.971521， $C_2 = 5.9\text{C}$ 为全样本最高第二阶段倍率。

典型短寿命策略：80PER_3_6C 与 4_8C_80PER_4_8C。

4.8 长、短寿命策略差异的初步解释

结合两阶段快充物理图像 [2,6,7]，可将差异归纳如下。

1. 高 SOC 区间高倍率。最短寿命策略在 $Q_1 = 31\%$ 之后仍以 5.9C 充电至 80% SOC。中高 SOC 下浓差极化、锂沉积风险与产热更显著，与题干“高 SOC 区间继续高倍率可能加剧老化”一致。
2. 低倍率、高 Q_1 并不自动等于长寿命。80PER_3_6C 充电时间长达约 13.4 min，但 SOH_{200} 仍偏低，说明“充得慢”不是充分保护条件，可能与长时间处于中高 SOC 或批次差异有关。
3. 同名参数、不同标签不可等同。4_8C_80PER_4_8C 与 4_8C_80PER_4_8C_NEWSTRUCTURE 的 C_1, Q_1, C_2 相同，但前者被标为短寿命、后者中位数落入长寿命。二者平均充电时间分别约为 10.06 min 与 11.04 min，说明实际过程或批次并不相同。NEWSTRUCTURE 应理解为后期实验框架标签，不是“充电条件完全相同”。
4. 样本与窗口局限。若干策略仅 2 块满 200 圈电池；早期三策略与 NEWSTRUCTURE 策略跨批次对比应降级为现象观察。正式因果推断与显著性检验留待问题二 [8,9]。

待完成 / 框架占位

问题一仍可补充：（1）策略参数总表（每策略 C_1, Q_1, C_2 与 IR、温度均值）；（2）NEWSTRUCTURE 内部单独排序作为公平对比主结论；（3）9 策略平均 SOH-cycle 曲线。上述不影响已给出的清洗、分布与典型策略结论。

5 问题二：充电策略及其参数对寿命衰减的影响

待完成 / 框架占位

本节为建模框架。仓库中尚无问题二的专用脚本与统计表。补全后应删除本框，填入检验统计量、回归系数与诊断图。

5.1 问题分析

问题一已表明策略间 SOH_{200} 存在可见差异，但 C_1, Q_1, C_2 并非完全独立变化，策略数仅 9、部分组 $n = 2$ 。因此本节重点是：组间差异是否具有统计显著性；三参数与寿命的相关/回归关系；不同 SOC 区间高倍率是否对应不同衰减特征；以及机制解释的局限性 [10–12]。

5.2 数据与可比性规则（拟采用）

1. 主分析：满 200 圈样本；因变量优先用 SOH_{200} 或 FadeRate。
2. 公平对比：NEWSTRUCTURE 内部互比作为主结论；带/不带该标签的同参数策略不作同条件重复实验。
3. 若纳入测试电池，统一截断至第 150 圈再比较，避免窗口不一致。

5.3 模型一：策略间差异的显著性检验

将策略视为分组因子。若各组近似正态且方差齐，采用单因素 ANOVA；否则采用 Kruskal–Wallis 检验，事后比较用 Tukey HSD 或 Dunn 检验，并报告效应量 η^2 [8]。

待填结果： H 或 F 统计量、 p 值、两两比较中显著的策略对。

5.4 模型二：参数与寿命的回归 / 相关分析

考察

$$\text{LifeProxy}_i = \beta_0 + \beta_1 C_{1,i} + \beta_2 Q_{1,i} + \beta_3 C_{2,i} + \beta_4 C_{2,i} \cdot \mathbf{1}\{Q_{1,i} < Q^*\} + \varepsilon_i. \quad (4)$$

因参数共线，可同时报告 Spearman 相关、方差膨胀因子（VIF），必要时采用混合效应（策略随机截距）[13]。交互项用于刻画“高倍率发生在低/高 SOC 区间”的差异。

待填结果：系数表、标准化系数或变量重要性、残差诊断。

5.5 不同 SOC 区间高倍率的衰减特征

将充电过程拆为低 SOC 段 $[0, Q_1]$ 与高 SOC 段 $[Q_1, 80\%]$ ，比较 C_1 主导段与 C_2 主导段对 FadeRate 的边际贡献。预期：在数据允许时， C_2 对寿命的负向作用强于同等幅度的 C_1 [2, 7]。

5.6 机制讨论与局限

拟从 SEI 生长、锂沉积、极化与产热等 LFP 相关路径讨论 [6, 14, 15]，并明确：策略非正交、早期衰减幅度小、cell-to-cell 变异与批次标签会限制因果识别。

6 问题三：基于早期循环的电池寿命预测

待完成 / 框架占位

本节为建模框架。9 块测试电池已在 `battery_summary.csv` 中由 `prediction_test=1` 标记，循环数据截至第 150 圈。预测表、误差指标与 EOL 外推结果待补入。

6.1 问题分析

实际中往往只能获得前期循环，无法等待完整老化。本题要求：用截至第 150 次循环的数据预测第 151–200 次循环 SOH，并进一步估计 $\text{SOH} = 80\%$ 的循环寿命；同时比较不同早期窗口长度，讨论策略信息与早期衰减特征的作用 [1, 12, 16–18]。

6.2 特征提取（拟采用）

表 4 问题三拟提取的特征类别

类别	示例
策略参数	C_1, Q_1, C_2 及策略编码
衰减趋势	前 k 圈 SOH 斜率、曲率、幂律/指数拟合参数
过程量	充电时间、IR、温度的均值与变化率
分段差	$\Delta\text{SOH}_{1\rightarrow 50}$ 、 $\Delta\text{SOH}_{50\rightarrow 100}$ 、 $\Delta\text{SOH}_{100\rightarrow 150}$

优先使用 `SOH_smooth`。若后续引入放电电压曲线类特征 $\Delta Q(V)$ ，需回源完整公开数据 [4, 16]。

6.3 预测模型（拟采用）

经验衰减模型（可解释）：

$$\text{SOH}(n) = 1 - a n^b \quad \text{或} \quad \text{SOH}(n) = a e^{-bn} + c, \quad (5)$$

用 1–150 圈拟合，外推 151–200 及 N_{EOL} 。

统计 / 机器学习模型：以早期统计特征预测后续 SOH 或衰减率，候选包括正则化线性模型、随机森林 / XGBoost、高斯过程；序列模型可试 LSTM [17]。验证建议：在 40 块非测试电池上做“截断 150 圈 → 预测 151–200”的交叉验证，再对 9 块测试电池给出最终预报，避免同策略信息泄漏。

6.4 评价指标与窗口比较

对 151–200 圈报告 RMSE、MAE 与 R^2 ；对 N_{EOL} 报告外推圈数。比较输入窗口 $k \in \{50, 100, 150\}$ 的精度差异。消融：有/无 (C_1, Q_1, C_2) 。

待填：9 块测试电池的 SOH 预测曲线、误差表、EOL 点估计。

7 问题四：兼顾充电时间与寿命衰减的策略优化

待完成 / 框架占位

本节为建模框架。优化应优先在已有 9 种实验策略内比较；若连续优化，参数须限制在实验数据合理邻域并说明依据 [19–22]。

7.1 决策变量与目标

决策变量 $x = (C_1, Q_1, C_2)$ 。目标为在较短充电时间下降低衰减速率或延长寿命：

$$\min_x F(x) = \begin{bmatrix} T_{\text{charge}}(x) \\ \text{DecayRate}(x) \end{bmatrix} \quad \text{或} \quad \min_x w_1 T_{\text{charge}}(x) + w_2 \text{DecayRate}(x). \quad (6)$$

7.2 充电时间模型（拟采用）

分段恒流近似

$$T_{\text{charge}} \approx \frac{Q_1}{C_1} + \frac{0.8 - Q_1}{C_2} + T_{\text{CV}}, \quad (7)$$

并用 `mean_chargetime` 回归校准。问题一散点图已显示充电时间与策略分组相关，可用于检验该近似。

7.3 寿命衰减模型

沿用问题二回归或问题三预测模型，得到 $\text{DecayRate}(x)$ 或 $\text{SOH}_{200}(x)$ 。在已有策略上先做离散 Pareto 比较，再考虑邻域连续搜索。

7.4 约束与邻域

建议参数盒约束取自样本极值（以最终校核为准）： $C_1 \in [3.6, 5.6]$ ， $Q_1 \in [19, 80]$ ， $C_2 \in [3.6, 5.9]$ 。超出实验凸包的组合须标明外推风险。

7.5 推荐策略与对比（待填）

给出推荐 (C_1^*, Q_1^*, C_2^*) ，并与问题一的典型长寿命策略 3_6C-80PER_3_6C、4_8C_80PER_4_8C_NEWSTRUCTURE 及短寿命策略 3_7C_31PER_5_9C_NEWSTRUCTURE 对比充电时间与寿命代理。说明适用范围（LFP、两阶段快充、早期循环窗口）与不可外推情形。

8 结论与展望

已完成结果（仓库现有成果）

问题一已形成的结论：

- 循环数据经 IQR 清洗后可用于策略对比；原始数据文件未改写。
- 在满 200 圈的 40 块电池中，第 200 次循环 SOH 均值为 0.9914，尚未达到 80% EOL。
- 按策略内 SOH 中位数，最长寿命策略为 3.6C(80%) – 3.6C，最短为 3.7C(31%) – 5.9C (NEWSTRUCTURE)。
- 高 SOC 区间维持高 C_2 的策略更容易出现更快衰减；同参数不同标签策略不可直接等同。

待完成 / 框架占位

补充问题二显著性与参数效应结论、问题三预测精度与 EOL 估计、问题四推荐策略及与长短寿命策略的对比；讨论模型局限与可推广范围。

参考文献

参考文献

- [1] Kristen A. Severson, Peter M. Attia, Norman Jin, Nicholas Perkins, Benben Jiang, Zi Yang, Michael H. Chen, Muratahan Aykol, Patrick K. Herring, Dimitrios Fraggedakis, Martin Z. Bazant, Stephen J. Harris, William C. Chueh, and Richard D. Braatz. Data-driven prediction of battery cycle life before capacity degradation. *Nature Energy*, 4(5):383–391, 2019. 本题主数据集来源；问题 1-4 必引.
- [2] Yayuan Liu, Yangying Zhu, and Yi Cui. Challenges and opportunities towards fast-charging battery materials. *Nature Energy*, 4:540–550, 2019. 快充机理综述；问题 1 机制解释.
- [3] Shabbir Ahmed, Ira Bloom, Andrew N. Jansen, Tanvir R. Tanim, Eric J. Dufek, Ahmad Pesaran, et al. Enabling fast charging – A battery technology gap assessment. *Journal of Power Sources*, 367:250–262, 2017. 快充技术缺口评估；问题 1/2 背景.
- [4] MIT–Stanford–TRI battery dataset. <https://data.matr.io/1/>. Severson 2019 配套公开数据；本题主数据.
- [5] Gregory L. Plett. *Battery Management Systems, Volume I: Battery Modeling*. Artech House, 2015. SOH/建模基础.
- [6] Matthieu Dubarry and Bor Yann Liaw. Identify capacity fading mechanism in a commercial LiFePO₄ cell. *Journal of Power Sources*, 194:541–549, 2009. LFP 衰减机理；问题 1/2.
- [7] Y. Ma et al. An aging-optimized state-of-charge-controlled multi-stage constant current (MCC) fast charging algorithm for commercial Li-ion battery based on three-electrode measurements. *Batteries*, 10(8):267, 2024. 多段恒流快充；问题 2/4.
- [8] Michael H. Kutner, Christopher J. Nachtsheim, John Neter, and William Li. *Applied Linear Statistical Models*. McGraw-Hill, 5 edition, 2005. 问题 2 ANOVA/回归.
- [9] Journal of The Electrochemical Society contributors. Fast charging of lithium-ion batteries while accounting for degradation and cell-to-cell variability. *Journal of The Electrochemical Society*, 2024. 快充、退化与单体差异；问题 2/4.
- [10] P. L. Ruiz et al. Physics-based and data-driven modeling of degradation mechanisms for lithium-ion batteries: A review. *Batteries*, 2023. TU Delft review; Problem 2.

- [11] H. Wang et al. High accuracy and applicability battery aging models for electric vehicle applications. *Journal of The Electrochemical Society*, 2024. 半经验老化模型；问题 2.
- [12] Alexis Geslin et al. Commentary on charging features for battery lifetime prediction. *Joule*, 2023. Charging-condition features and lifetime prediction; Problem 2/3.
- [13] José Pinheiro and Douglas Bates. *Mixed-Effects Models in S and S-PLUS*. Springer, 2000. 问题 2 混合效应模型.
- [14] Matthieu Dubarry, Cyril Truchot, and Bor Yann Liaw. Synthesize battery degradation modes via a diagnostic and prognostic model. *Journal of Power Sources*, 219:204–216, 2012. 退化模式诊断框架；问题 1/2.
- [15] W. Li et al. Review of state-of-the-art degradation models for lithium-ion batteries. *Entropy*, 28(6):669, 2026. 退化模型新综述；问题 2.
- [16] Peter M. Attia, Aditya Grover, Norman Jin, Kristen A. Severson, et al. Statistical learning for accurate and interpretable battery lifetime prediction. *Journal of The Electrochemical Society*, 2021. 问题 3 特征工程与可解释预测.
- [17] Yongzhi Zhang, Rui Xiong, Hongwen He, and Michael G. Pecht. Long short-term memory recurrent neural network for remaining useful life prediction of lithium-ion batteries. *IEEE Transactions on Vehicular Technology*, 67(7):5695–5705, 2018. LSTM-RNN and Monte Carlo probabilistic RUL prediction.
- [18] Yongzhi Zhang, Rui Xiong, Hongwen He, and Michael G. Pecht. Lithium-ion battery remaining useful life prediction with box-cox transformation and monte carlo simulation. *IEEE Transactions on Industrial Electronics*, 66(2):1585–1597, 2019. Box-Cox transformation and Monte Carlo simulation for online RUL prediction.
- [19] Peter M. Attia, Aditya Grover, Norman Jin, Kristen A. Severson, Todor M. Markov, Yang Liao, Michael H. Chen, Bryan Cheong, Nicholas Perkins, Zi Yang, Patrick K. Herring, Muratahan Aykol, Stephen J. Harris, Richard D. Braatz, Stefano Ermon, and William C. Chueh. Closed-loop optimization of fast-charging protocols for batteries with machine learning. *Nature*, 578:397–402, 2020. 问题 3 早期预测 + 问题 4 贝叶斯优化快充协议.
- [20] Gregory L. Plett and M. Scott Trimboli. *Battery Management Systems, Volume III: Physics-Based Methods*. Artech House, 2022. 物理模型与快充优化；问题 4.
- [21] Peter I. Frazier. A tutorial on Bayesian optimization, 2018. 问题 4 贝叶斯优化方法.

[22] Kalyanmoy Deb. *Multi-Objective Optimization Using Evolutionary Algorithms*. Wiley, 2001. 问题 4 多目标优化 / NSGA-II.

A 支撑材料文件列表

按赛制，支撑材料单独打包为压缩文件（RAR/ZIP，不超过 20 MB），本附录列出其内容。赛题原始数据 `data/battery_summary.csv`、`data/cycle_train.csv` 不作为“自主查阅数据”重复提交，但建模程序依赖它们运行。

表 5 支撑材料拟包含的文件

路径	说明
<code>src/A/task1_pipeline.py</code>	问题一完整可运行流水线
<code>src/A/SOH_plot.py</code>	SOH 曲线绘制模块
<code>A/问题一/AGENTS.md</code>	问题一任务说明（提示词）
<code>A/问题一/log_1.md</code> 、 <code>log_2.md</code>	运行日志
<code>output/A/cycle_train_cleaned.csv</code>	IQR 清洗后循环数据
<code>output/A/charge_policy.csv</code>	电池-策略表
<code>output/A/battery_SOH_charge.csv</code>	200 圈 SOH 与充电时间
<code>output/A/policy.csv</code>	策略汇总与寿命标签
<code>output/A/doc/1.md</code>	典型长/短寿命策略说明
<code>output/A/*.png</code>	问题一图件
<code>output/A/1/</code>	问题一第一轮归档
<code>references/</code>	参考文献导读与 BibTeX
<code>AI 使用记录/</code>	AI 使用详情底稿
<code>paper/</code>	本 LaTeX 工程

问题二至问题四的脚本、中间表与预测结果待生成后追加本表。若最终提交时上述文件均已纳入压缩包，则与论文结论对应；源程序运行方式见附录 B。

B 建模源程序

本节给出问题一全部可运行源代码。运行环境：Python 3，依赖 `numpy`、`pandas`、`matplotlib`。在项目根目录执行：

```
1 python src/A/task1_pipeline.py
```

可选参数 `--iqr-factor`（默认 1.5）。单独绘制指定电池 SOH 曲线：

```
1 python src/A/SOH_plot.py --battery-id 1
```

待完成 / 框架占位

问题二–四源程序完成后，以同样方式用 `\lstinputlisting` 追加完整代码，确保运行结果与正文图表一致。

B.1 SOH_plot.py

Listing 1 SOH 曲线绘制脚本

```
1  """绘制指定电池的 SOH 曲线。"""
2
3  from __future__ import annotations
4
5  import argparse
6  from pathlib import Path
7
8  import matplotlib
9
10 matplotlib.use("Agg")
11
12 import matplotlib.font_manager as font_manager
13 import matplotlib.pyplot as plt
14 import pandas as pd
15
16
17 PROJECT_ROOT = Path(__file__).resolve().parents[2]
18
19
20 def configure_chinese_font() -> str:
21     """选择系统中可用的中文字体，并返回字体名称。"""
22     installed = {font.name for font in font_manager.fontManager.ttflist}
23     candidates = [
24         "Microsoft YaHei",
25         "SimHei",
26         "Noto Sans CJK SC",
27         "Source Han Sans SC",
28         "Arial Unicode MS",
29     ]
30     selected = next((name for name in candidates if name in installed), "DejaVu Sans")
31     plt.rcParams["font.sans-serif"] = [selected, "DejaVu Sans"]
32     plt.rcParams["axes.unicode_minus"] = False
33     return selected
34
35
36 def plot_battery_soh(
37     cycles: pd.DataFrame,
38     battery_id: int,
39     output_path: Path,
40 ) -> None:
```



```

41 """将指定电池的原始 SOH 与平滑 SOH 绘制到同一张图。"""
42 required = {"battery_id", "cycle", "SOH", "SOH_smooth"}
43 missing = required.difference(cycles.columns)
44 if missing:
45     raise ValueError(f"循环数据缺少字段: {sorted(missing)}")
46
47 battery = cycles.loc[cycles["battery_id"] == battery_id].sort_values("cycle"
48 )
49 if battery.empty:
50     raise ValueError(f"未找到 battery_id={battery_id} 的数据")
51
52 configure_chinese_font()
53 fig, ax = plt.subplots(figsize=(10, 6))
54 ax.plot(
55     battery["cycle"],
56     battery["SOH"],
57     color="#4C78A8",
58     linewidth=1.2,
59     alpha=0.65,
60     label="SOH (清洗后)",
61 )
62 ax.plot(
63     battery["cycle"],
64     battery["SOH_smooth"],
65     color="#F28E2B",
66     linewidth=2.2,
67     label="平滑 SOH",
68 )
69 ax.set_title(f"{battery_id} 号电池 SOH 曲线")
70 ax.set_xlabel("循环次数")
71 ax.set_ylabel("电池健康状态 (SOH)")
72 ax.grid(True, color="#D9D9D9", linewidth=0.7, alpha=0.7)
73 ax.legend(loc="upper right")
74 fig.tight_layout()
75 output_path.parent.mkdir(parents=True, exist_ok=True)
76 fig.savefig(output_path, dpi=300, bbox_inches="tight")
77 plt.close(fig)
78
79 def parse_args() -> argparse.Namespace:
80     parser = argparse.ArgumentParser(description="绘制指定电池的 SOH 曲线")
81     parser.add_argument("--battery-id", type=int, default=1, help="电池编号, 默认
82         为 1")
83     parser.add_argument(

```

```

83     "--input",
84     type=Path,
85     default=PROJECT_ROOT / "output" / "A" / "cycle_train_cleaned.csv",
86     help="循环数据 CSV 路径",
87 )
88 parser.add_argument(
89     "--output",
90     type=Path,
91     default=PROJECT_ROOT / "output" / "A" / "SOH_1_example.png",
92     help="输出图像路径",
93 )
94 return parser.parse_args()
95
96
97 def main() -> None:
98     args = parse_args()
99     cycles = pd.read_csv(args.input)
100    plot_battery_soh(cycles, args.battery_id, args.output)
101    print(f"已生成: {args.output}")
102
103
104 if __name__ == "__main__":
105     main()

```

B.2 task1_pipeline.py

Listing 2 问题一分析流水线

```

1  """问题一：数据清洗、策略提取、200 次循环汇总与绘图。"""
2
3  from __future__ import annotations
4
5  import argparse
6  from collections import Counter
7  from datetime import datetime
8  from pathlib import Path
9
10 import matplotlib
11
12 matplotlib.use("Agg")
13
14 import matplotlib.pyplot as plt
15 from matplotlib.patches import Patch
16 import numpy as np

```

```

17 import pandas as pd
18
19 from SOH_plot import configure_chinese_font, plot_battery_soh
20
21
22 PROJECT_ROOT = Path(__file__).resolve().parents[2]
23 MEASUREMENT_COLUMNS = ["capacity", "SOH", "SOH_smooth", "chargetime", "IR", "
    Tavg"]
24
25
26 def validate_inputs(summary: pd.DataFrame, cycles: pd.DataFrame) -> None:
27     summary_required = {"battery_id", "policy", "initial_capacity", "
        prediction_test"}
28     cycle_required = {
29         "battery_id",
30         "cycle",
31         "capacity",
32         "SOH",
33         "SOH_smooth",
34         "chargetime",
35         "IR",
36         "Tavg",
37         "policy",
38     }
39     summary_missing = summary_required.difference(summary.columns)
40     cycle_missing = cycle_required.difference(cycles.columns)
41     if summary_missing or cycle_missing:
42         raise ValueError(
43             f"输入字段不完整; battery_summary 缺少 {sorted(summary_missing)}, "
44             f"cycle_train 缺少 {sorted(cycle_missing)}"
45         )
46     if summary["battery_id"].duplicated().any():
47         raise ValueError("battery_summary.csv 中 battery_id 不唯一")
48     if cycles.duplicated(["battery_id", "cycle"]).any():
49         raise ValueError("cycle_train.csv 中存在重复的 battery_id + cycle")
50     if set(summary["battery_id"]) != set(cycles["battery_id"]):
51         raise ValueError("两个输入文件的 battery_id 集合不一致")
52
53     policy_check = cycles.merge(
54         summary[["battery_id", "policy"]],
55         on="battery_id",
56         suffixes=("_cycle", "_summary"),
57         validate="many_to_one",
58     )

```

```

59     if (policy_check["policy_cycle"] != policy_check["policy_summary"]).any():
60         raise ValueError("cycle_train.csv 与 battery_summary.csv 的策略字段不一致")
61
62
63     def iqr_clean_by_battery(
64         cycles: pd.DataFrame,
65         columns: list[str],
66         factor: float = 1.5,
67     ) -> tuple[pd.DataFrame, list[dict[str, float | int | str]]]:
68         """逐电池、逐字段进行 IQR 检测，并用最近前后正常值的均值替换。"""
69         cleaned = cycles.sort_values(["battery_id", "cycle"]).copy()
70         audit: list[dict[str, float | int | str]] = []
71
72         for battery_id, group in cycles.sort_values(["battery_id", "cycle"]).groupby(
73             "battery_id"):
74             for column in columns:
75                 values = group[column].to_numpy(dtype=float)
76                 finite_values = values[np.isfinite(values)]
77                 if finite_values.size == 0:
78                     continue
79
80                 q1, q3 = np.quantile(finite_values, [0.25, 0.75])
81                 iqr = q3 - q1
82                 if np.isclose(iqr, 0.0):
83                     continue
84
85                 lower = q1 - factor * iqr
86                 upper = q3 + factor * iqr
87                 outlier_mask = (~np.isfinite(values)) | (values < lower) | (values >
88                     upper)
89                 normal_positions = np.flatnonzero(~outlier_mask)
90
91                 for position in np.flatnonzero(outlier_mask):
92                     left_candidates = normal_positions[normal_positions < position]
93                     right_candidates = normal_positions[normal_positions > position]
94                     neighbors: list[float] = []
95                     if left_candidates.size:
96                         neighbors.append(float(values[left_candidates[-1]]))
97                     if right_candidates.size:
98                         neighbors.append(float(values[right_candidates[0]]))
99                     if not neighbors:
100                         continue
101
102                     replacement = float(np.mean(neighbors))

```

```

100         row_index = group.index[position]
101         cleaned.at[row_index, column] = replacement
102         audit.append(
103             {
104                 "battery_id": int(battery_id),
105                 "cycle": int(group.iloc[position]["cycle"]),
106                 "column": column,
107                 "original": float(values[position]),
108                 "replacement": replacement,
109                 "lower_bound": float(lower),
110                 "upper_bound": float(upper),
111             }
112         )
113
114     return cleaned.sort_values(["battery_id", "cycle"]), audit
115
116
117 def build_policy_table(summary: pd.DataFrame) -> pd.DataFrame:
118     return (
119         summary[["battery_id", "policy"]]
120         .sort_values("battery_id")
121         .reset_index(drop=True)
122     )
123
124
125 def build_cycle_200_summary(
126     cleaned_cycles: pd.DataFrame,
127     policy_table: pd.DataFrame,
128 ) -> pd.DataFrame:
129     max_cycle = cleaned_cycles.groupby("battery_id")["cycle"].max()
130     eligible_ids = max_cycle[max_cycle == 200].index
131     eligible = cleaned_cycles[cleaned_cycles["battery_id"].isin(eligible_ids)]
132
133     cycle_200 = eligible.loc[eligible["cycle"] == 200, ["battery_id", "SOH"]]
134     if cycle_200["battery_id"].duplicated().any() or len(cycle_200) != len(
135         eligible_ids):
136         raise ValueError("达到 200 次循环的电池没有且仅有一条 cycle=200 记录")
137
138     charge_mean = (
139         eligible.groupby("battery_id", as_index=False)["charge_time"]
140         .mean()
141         .rename(columns={"charge_time": "charge_time_mean"})
142     )
143     result = (

```

```

143     cycle_200.merge(policy_table, on="battery_id", validate="one_to_one")
144     .merge(charge_mean, on="battery_id", validate="one_to_one")
145     [["battery_id", "policy", "SOH", "charge_time_mean"]]
146     .sort_values("battery_id")
147     .reset_index(drop=True)
148 )
149 return result
150
151
152 def build_policy_summary(
153     data: pd.DataFrame,
154     policy_order: list[str],
155     critical_soh_low: float,
156     critical_soh_high: float,
157 ) -> pd.DataFrame:
158     """汇总各充电策略，并用策略 SOH 中位数划分寿命类型。"""
159     classified = data.assign(
160         lifetime_group=np.select(
161             [data["SOH"] >= critical_soh_high, data["SOH"] <= critical_soh_low],
162             ["长寿命", "短寿命"],
163             default="中等寿命",
164         )
165     )
166     grouped = classified.groupby("policy", sort=False)
167     result = grouped.agg(
168         battery_count=("battery_id", "size"),
169         SOH_min=("SOH", "min"),
170         SOH_max=("SOH", "max"),
171         SOH_mean=("SOH", "mean"),
172         SOH_median=("SOH", "median"),
173         charge_time_min=("charge_time_mean", "min"),
174         charge_time_max=("charge_time_mean", "max"),
175     ).reindex(policy_order)
176     group_counts = (
177         classified.groupby(["policy", "lifetime_group"])
178         .size()
179         .unstack(fill_value=0)
180         .reindex(policy_order, fill_value=0)
181     )
182     for group_name, column_name in [
183         ("长寿命", "long_battery_count"),
184         ("中等寿命", "middle_battery_count"),
185         ("短寿命", "short_battery_count"),
186     ]:

```

```

187         result[column_name] = group_counts.get(group_name, pd.Series(0, index=
188             result.index)).astype(int)
189
190     result["lifetime_label"] = np.select(
191         [result["SOH_median"] >= critical_soh_high, result["SOH_median"] <=
192             critical_soh_low],
193         ["长寿命策略", "短寿命策略"],
194         default="中等寿命策略",
195     )
196     max_median = result["SOH_median"].max()
197     min_median = result["SOH_median"].min()
198     result["longest_strategy"] = np.where(np.isclose(result["SOH_median"],
199         max_median), "是", "否")
200     result["shortest_strategy"] = np.where(np.isclose(result["SOH_median"],
201         min_median), "是", "否")
202     result["SOH_range"] = result.apply(lambda row: f"{row['SOH_min']:.6f} - {row
203         ['SOH_max']:.6f}", axis=1)
204     result["charge_time_range"] = result.apply(
205         lambda row: f"{row['charge_time_min']:.6f} - {row['charge_time_max']:.6f}"
206         , axis=1
207     )
208     return result.reset_index()
209
210 def save_soh_histogram(
211     data: pd.DataFrame,
212     output_path: Path,
213     critical_soh_low: float,
214     critical_soh_high: float,
215 ) -> None:
216     values = data["SOH"].to_numpy(dtype=float)
217     bin_edges = np.histogram_bin_edges(values, bins="fd")
218     if len(bin_edges) < 7:
219         bin_edges = np.linspace(values.min(), values.max(), 8)
220
221     fig, ax = plt.subplots(figsize=(10, 6))
222     ax.hist(
223         values,
224         bins=bin_edges,
225         color="#4C78A8",
226         edgecolor="white",
227         linewidth=1.0,
228         label="电池数量",
229     )

```

```

225     ax.axvline(
226         critical_soh_high,
227         color="#D62728",
228         linestyle="--",
229         linewidth=2,
230         label=f"长寿命分界线 (75%分位)={critical_soh_high:.4f}",
231     )
232     ax.axvline(
233         critical_soh_low,
234         color="#1F77B4",
235         linestyle="--",
236         linewidth=2,
237         label=f"短寿命分界线 (25%分位)={critical_soh_low:.4f}",
238     )
239     margin = max((values.max() - values.min()) * 0.08, 0.003)
240     ax.set_xlim(values.min() - margin, values.max() + margin)
241     ax.set_title("第 200 次循环 SOH 分布")
242     ax.set_xlabel("第 200 次循环的 SOH")
243     ax.set_ylabel("电池数量")
244     ax.grid(axis="y", color="#D9D9D9", linewidth=0.7, alpha=0.7)
245     ax.legend(loc="upper right")
246     fig.tight_layout()
247     fig.savefig(output_path, dpi=300, bbox_inches="tight")
248     plt.close(fig)
249
250
251 def save_soh_charge_scatter(
252     data: pd.DataFrame,
253     output_path: Path,
254     policy_order: list[str],
255 ) -> None:
256     fig, ax = plt.subplots(figsize=(13, 8))
257     color_map = matplotlib.colormaps.get_cmap("tab10")
258     colors = color_map(np.linspace(0, 1, max(len(policy_order), 2)))
259     for color, policy in zip(colors, policy_order):
260         group = data.loc[data["policy"] == policy]
261         if group.empty:
262             continue
263         ax.scatter(
264             group["SOH"],
265             group["charge_time_mean"],
266             s=64,
267             color=color,
268             edgecolors="white",

```



```

269         linewidths=0.8,
270         alpha=0.88,
271         label=policy,
272     )
273     ax.set_title("第 200 次循环 SOH 与平均充电时间")
274     ax.set_xlabel("第 200 次循环的 SOH")
275     ax.set_ylabel("平均充电时间 (min) ")
276     ax.grid(True, color="#D9D9D9", linewidth=0.7, alpha=0.7)
277     ax.legend(loc="upper right", fontsize=8, framealpha=0.92, title="充电策略",
278             title_fontsize=9)
279     fig.tight_layout()
280     fig.savefig(output_path, dpi=300, bbox_inches="tight")
281     plt.close(fig)
282
283 def save_policy_boxplot(
284     data: pd.DataFrame,
285     output_path: Path,
286     critical_soh_low: float,
287     critical_soh_high: float,
288     policy_order: list[str],
289 ) -> None:
290     available_order = [policy for policy in policy_order if policy in set(data["
291         policy"])]
292     groups = [data.loc[data["policy"] == policy, "SOH"].to_numpy() for policy in
293         available_order]
294
295     fig, ax = plt.subplots(figsize=(16, 8))
296     box = ax.boxplot(
297         groups,
298         tick_labels=available_order,
299         patch_artist=True,
300         widths=0.62,
301         medianprops={"color": "#1F1F1F", "linewidth": 1.5},
302         whiskerprops={"color": "#555555"},
303         capprops={"color": "#555555"},
304         flierprops={
305             "marker": "o",
306             "markerfacecolor": "white",
307             "markeredgecolor": "#4C78A8",
308             "markersize": 5,
309         },
310     )
311     for patch in box["boxes"]:

```

```

310     patch.set_facecolor("#9ECAE1")
311     patch.set_edgecolor("#4C78A8")
312
313     ax.axhline(
314         critical_soh_high,
315         color="#D62728",
316         linestyle="--",
317         linewidth=2,
318     )
319     ax.axhline(
320         critical_soh_low,
321         color="#1F77B4",
322         linestyle="--",
323         linewidth=2,
324     )
325     all_values = data["SOH"].to_numpy(dtype=float)
326     margin = max((all_values.max() - all_values.min()) * 0.08, 0.003)
327     ax.set_ylim(all_values.min() - margin, all_values.max() + margin)
328     ax.set_title("不同充电策略下的第 200 次循环 SOH 分布")
329     ax.set_xlabel("充电策略")
330     ax.set_ylabel("使用寿命指标: 第 200 次循环的 SOH")
331     ax.tick_params(axis="x", labelrotation=40)
332     for label in ax.get_xticklabels():
333         label.set_horizontalalignment("right")
334     ax.grid(axis="y", color="#D9D9D9", linewidth=0.7, alpha=0.7)
335     legend_handles = [
336         Patch(facecolor="#9ECAE1", edgecolor="#4C78A8", label="策略内 SOH 分布"),
337         plt.Line2D(
338             [0], [0], color="#D62728", linestyle="--", linewidth=2,
339             label=f"长寿命分界线={critical_soh_high:.4f}",
340         ),
341         plt.Line2D(
342             [0], [0], color="#1F77B4", linestyle="--", linewidth=2,
343             label=f"短寿命分界线={critical_soh_low:.4f}",
344         ),
345     ]
346     ax.legend(handles=legend_handles, loc="upper right")
347     fig.tight_layout()
348     fig.savefig(output_path, dpi=300, bbox_inches="tight")
349     plt.close(fig)
350
351
352 def write_typical_policy_doc(
353     doc_path: Path,

```

```

354 policy_summary: pd.DataFrame,
355 critical_soh_low: float,
356 critical_soh_high: float,
357 ) -> None:
358     long_policies = policy_summary.loc[policy_summary["lifetime_label"] == "长寿命策略"]
359     short_policies = policy_summary.loc[policy_summary["lifetime_label"] == "短寿命策略"]
360     longest = policy_summary.loc[policy_summary["longest_strategy"] == "是"].iloc[0]
361     shortest = policy_summary.loc[policy_summary["shortest_strategy"] == "是"].iloc[0]
362
363     lines = [
364         "# 典型长寿命与短寿命充电策略",
365         "",
366         "本分析使用第 200 次循环的 SOH 作为早期使用寿命代理指标。",
367         f"长寿命阈值为样本 SOH 的 75% 分位数 ({critical_soh_high:.6f}) , ",
368         f"短寿命阈值为 25% 分位数 ({critical_soh_low:.6f})。策略类型按策略内 SOH 中位数判定。",
369         "",
370         "## 关键结论",
371         "",
372         f"- **寿命最长策略: `{longest['policy']}`**, SOH 中位数 {longest['SOH_median']:.6f}, 范围 {longest['SOH_range']}。",
373         f"- **寿命最短策略: `{shortest['policy']}`**, SOH 中位数 {shortest['SOH_median']:.6f}, 范围 {shortest['SOH_range']}。",
374         "",
375         "## 典型长寿命策略",
376         "",
377         "| 策略 | 电池数 | SOH中位数 | SOH范围 | 平均充电时间范围/min | 标记 |",
378         "|---|---:|---:|---|---|---|",
379     ]
380     for _, row in long_policies.sort_values("SOH_median", ascending=False).iterrows():
381         mark = "寿命最长" if row["longest_strategy"] == "是" else "典型长寿命"
382         lines.append(
383             f"| `{row['policy']}` | {int(row['battery_count'])} | {row['SOH_median']:.6f} | "
384             f"{row['SOH_range']} | {row['charge_time_range']} | **{mark}** |"
385         )
386     lines.extend(
387         [
388             "",

```

```

389         "## 典型短寿命策略",
390         "",
391         "| 策略 | 电池数 | SOH中位数 | SOH范围 | 平均充电时间范围/min | 标记 |"
392         ,
393         "|---|---:|---:|---|---|---|",
394     ]
395 )
396 for _, row in short_policies.sort_values("SOH_median").iterrows():
397     mark = "寿命最短" if row["shortest_strategy"] == "是" else "典型短寿命"
398     lines.append(
399         f"| `{row['policy']}` | {int(row['battery_count'])} | {row['SOH_median']:.6f} | "
400         f"{row['SOH_range']} | {row['charge_time_range']} | **{mark}** |"
401     )
402     lines.extend(
403         [
404             "",
405             "## 解释限制",
406             "",
407             "- 各策略的完整 200 循环样本数较少，结论用于本数据集内的典型策略比较，不宜直接外推为因果结论。",
408             "- `4_8C_80PER_4_8C_NEWSTRUCTURE` 的中位数较高但包含一个明显较低的 SOH 个体，需结合离散程度解读。",
409             "- 数据未覆盖 SOH<80% 的真实寿命终止循环，因此这里比较的是第 200 次循环的健康保持程度。",
410         ]
411     )
412     doc_path.parent.mkdir(parents=True, exist_ok=True)
413     doc_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
414
415 def write_csv_if_changed(data: pd.DataFrame, path: Path) -> None:
416     """内容不变时避免重写，兼容 CSV 正被表格软件打开的情形。"""
417     if path.exists():
418         try:
419             existing = pd.read_csv(path)
420             pd.testing.assert_frame_equal(existing, data, check_dtype=False)
421             return
422         except (AssertionError, OSError, ValueError):
423             pass
424     data.to_csv(path, index=False, encoding="utf-8-sig")
425
426 def write_log(

```

```

428     log_path: Path,
429     summary: pd.DataFrame,
430     cycles: pd.DataFrame,
431     audit: list[dict[str, float | int | str]],
432     cycle_200: pd.DataFrame,
433     policy_summary: pd.DataFrame,
434     critical_soh_low: float,
435     critical_soh_high: float,
436     iqr_factor: float,
437     font_name: str,
438     output_paths: list[Path],
439     prompt_text: str,
440 ) -> None:
441     counts = Counter(str(item["column"]) for item in audit)
442     max_cycle_counts = cycles.groupby("battery_id")["cycle"].max().value_counts
443         ().sort_index()
444     missing_summary = int(summary.isna().sum().sum())
445     missing_cycles = int(cycles.isna().sum().sum())
446     long_count = int((cycle_200["SOH"] >= critical_soh_high).sum())
447     short_count = int((cycle_200["SOH"] <= critical_soh_low).sum())
448     middle_count = len(cycle_200) - long_count - short_count
449     longest = policy_summary.loc[policy_summary["longest_strategy"] == "是", "
        policy"].iloc[0]
450     shortest = policy_summary.loc[policy_summary["shortest_strategy"] == "是", "
        policy"].iloc[0]
451
452     lines = [
453         "# 问题一第二次迭代运行日志 (log_2) ",
454         "",
455         f"- 运行时间: {datetime.now().astimezone().isoformat(timespec='seconds')}"
456         "",
457         f"- IQR 系数: {iqr_factor:.6g}",
458         f"- critical_SOH_high (75%分位数): {critical_soh_high:.6f}",
459         f"- critical_SOH_low (25%分位数): {critical_soh_low:.6f}",
460         f"- 绘图字体: {font_name}",
461         "",
462         "## 输入质量检查",
463         "",
464         f"- battery_summary.csv: {len(summary)} 行, {summary.shape[1]} 列, {
            summary['battery_id'].nunique()} 块电池。",
465         f"- cycle_train.csv: {len(cycles)} 行, {cycles.shape[1]} 列。",
466         f"- battery_id + cycle 重复记录: {int(cycles.duplicated(['battery_id', '
            cycle']).sum())}。",
467         f"- 汇总表缺失单元格: {missing_summary} (其中 C1 的结构性缺失保留, 不参与循

```

```

    环数据清洗)。",
466 f"- 循环表缺失单元格: {missing_cycles}。",
467 f"- 最大循环数分布: {dict((int(k), int(v)) for k, v in max_cycle_counts.
    items())}。",
468 "",
469 "## IQR 清洗",
470 "",
471 "- 方法: 按 battery_id 分组, 对 capacity、SOH、SOH_smooth、chargetime、IR
    、Tavg 分别计算 Q1、Q3 和 IQR。",
472 "- 判定: 小于  $Q1 - 1.5 \times IQR$  或大于  $Q3 + 1.5 \times IQR$  的值标记为异常。",
473 "- 填补: 使用同一电池、同一字段中距离最近的前后正常值均值; 边界点仅有一侧正
    常值时使用该侧值。",
474 f"- 共替换 {len(audit)} 个单元格; 分字段数量: {dict(counts)}。",
475 "- 原始 data 文件未改写; 清洗结果另存为 output/A/cycle_train_cleaned.csv。
    ",
476 "",
477 "## 200 次循环样本",
478 "",
479 f"- 纳入: 最大 cycle 恰为 200 的 {len(cycle_200)} 块电池。",
480 f"- 排除: 仅记录至 150 次循环的 {summary['prediction_test'].sum()} 块测试
    电池。",
481 f"- SOH 范围: {cycle_200['SOH'].min():.6f} - {cycle_200['SOH'].max():.6f}
    , 均值 {cycle_200['SOH'].mean():.6f}。",
482 f"- 按两个分位数分类: 长寿命 {long_count} 块, 中等寿命 {middle_count} 块,
    短寿命 {short_count} 块。",
483 f"- 寿命最长策略 (按策略 SOH 中位数): {longest}。",
484 f"- 寿命最短策略 (按策略 SOH 中位数): {shortest}。",
485 "",
486 "## 第一轮归档",
487 "",
488 "- 第一轮输出已整理到 `output/A/1/`, 共 7 个交付文件。",
489 "",
490 "## 输出文件",
491 "",
492 ]
493 lines.extend(f"- `{path.relative_to(PROJECT_ROOT).as_posix()}`" for path in
    output_paths)
494 lines.extend(
495 [
496     "",
497     "## 说明",
498     "",
499     "- 箱型图纵轴采用“第 200 次循环 SOH”作为使用寿命代理指标; 本题数据尚未
        覆盖 SOH<80% 的真实寿命终止循环。",

```

```

500         "- IQR 是统计异常筛查，不等同于确认传感器故障；清洗结果用于完成本题指定
           分析。",
501         "- 策略寿命标签按策略内 SOH 中位数与总体 25%/75% 分位阈值比较得到。",
502         "",
503         "## 原始提示词",
504         "",
505         "以下完整保留本次执行时 `A/问题一/AGENTS.md` 的内容：",
506         "",
507         "``text",
508         prompt_text.rstrip(),
509         "``",
510     ]
511 )
512 log_path.parent.mkdir(parents=True, exist_ok=True)
513 log_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
514
515
516 def parse_args() -> argparse.Namespace:
517     parser = argparse.ArgumentParser(description="运行问题一完整分析流程")
518     parser.add_argument("--iqr-factor", type=float, default=1.5, help="IQR 异常
           判定系数，默认 1.5")
519     return parser.parse_args()
520
521
522 def main() -> None:
523     args = parse_args()
524     if args.iqr_factor <= 0:
525         raise ValueError("--iqr-factor 必须大于 0")
526
527     data_dir = PROJECT_ROOT / "data"
528     output_dir = PROJECT_ROOT / "output" / "A"
529     output_dir.mkdir(parents=True, exist_ok=True)
530     summary = pd.read_csv(data_dir / "battery_summary.csv")
531     cycles = pd.read_csv(data_dir / "cycle_train.csv")
532     validate_inputs(summary, cycles)
533
534     cleaned, audit = iqr_clean_by_battery(cycles, MEASUREMENT_COLUMNS, args.
           iqr_factor)
535     policy_table = build_policy_table(summary)
536     cycle_200 = build_cycle_200_summary(cleaned, policy_table)
537     critical_soh_low = float(cycle_200["SOH"].quantile(0.25))
538     critical_soh_high = float(cycle_200["SOH"].quantile(0.75))
539     policy_order = list(dict.fromkeys(summary.sort_values("battery_id")["policy"]
           ))

```

```

540 policy_summary = build_policy_summary(
541     cycle_200,
542     policy_order,
543     critical_soh_low,
544     critical_soh_high,
545 )
546
547 cleaned_path = output_dir / "cycle_train_cleaned.csv"
548 policy_path = output_dir / "charge_policy.csv"
549 summary_path = output_dir / "battery_SOH_charge.csv"
550 soh_example_path = output_dir / "SOH_1_example.png"
551 histogram_path = output_dir / "SOH_summary.png"
552 scatter_path = output_dir / "SOH_chargetime.png"
553 boxplot_path = output_dir / "charge_policy.png"
554 policy_summary_path = output_dir / "policy.csv"
555 typical_doc_path = output_dir / "doc" / "1.md"
556
557 cleaned.to_csv(cleaned_path, index=False, encoding="utf-8-sig")
558 write_csv_if_changed(policy_table, policy_path)
559 cycle_200.to_csv(summary_path, index=False, encoding="utf-8-sig")
560 policy_summary.to_csv(policy_summary_path, index=False, encoding="utf-8-sig")
561
562 font_name = configure_chinese_font()
563 plot_battery_soh(cleaned, 1, soh_example_path)
564 save_soh_histogram(cycle_200, histogram_path, critical_soh_low,
565     critical_soh_high)
566 save_soh_charge_scatter(cycle_200, scatter_path, policy_order)
567 save_policy_boxplot(
568     cycle_200,
569     boxplot_path,
570     critical_soh_low,
571     critical_soh_high,
572     policy_order,
573 )
574 write_typical_policy_doc(
575     typical_doc_path,
576     policy_summary,
577     critical_soh_low,
578     critical_soh_high,
579 )
580
581 output_paths = [
582     cleaned_path,

```



```

582     policy_path,
583     summary_path,
584     soh_example_path,
585     histogram_path,
586     scatter_path,
587     boxplot_path,
588     policy_summary_path,
589     typical_doc_path,
590 ]
591 write_log(
592     PROJECT_ROOT / "A" / "问题一" / "log_2.md",
593     summary,
594     cycles,
595     audit,
596     cycle_200,
597     policy_summary,
598     critical_soh_low,
599     critical_soh_high,
600     args.iqr_factor,
601     font_name,
602     output_paths,
603     (PROJECT_ROOT / "A" / "问题一" / "AGENTS.md").read_text(encoding="utf-8")
604     ,
605 )
606
607 print(f"IQR 替换单元格数: {len(audit)}")
608 print(f"200 次循环样本数: {len(cycle_200)}")
609 print(f"critical_SOH_low: {critical_soh_low:.6f}")
610 print(f"critical_SOH_high: {critical_soh_high:.6f}")
611 for path in output_paths:
612     print(f"已生成: {path}")
613
614 if __name__ == "__main__":
615     main()

```

C AI 工具使用声明

根据赛事论文格式规范，现就人工智能工具使用情况作如下声明。全文不出现参赛者姓名与学校信息。

C.1 所用工具

- OpenAI ChatGPT（记录版本：1.2026.190，模型 gpt5.6-sol-high）
- OpenAI Codex CLI（记录版本：v0.147.0，模型 gpt5.6-sol-high；DeepSeek-V4-Flash-0731）
- 对话式编程助手（用于文献整理、问题一结果写入 LaTeX 论文框架）

C.2 使用目的与环节

1. 讨论与协作：询问 Git 多人协作与仓库管理相关操作说明。
2. 问题一：依据人工撰写的 AGENTS.md 生成模块化分析代码，完成 IQR 清洗、策略提取、200 圈汇总与绘图；人工检查可运行性后调整参数并复现日志。
3. 文献：整理快充、SOH 预测与策略优化相关公开文献条目，写入 references/。
4. 论文半成品：将仓库已有问题一成果写入 LaTeX 正文，并为问题二至四保留章节框架、符号与拟采用方法。

C.3 提示方式

主要方式为：先由人工完成题意分析与方法构想，再将任务写成 AGENTS.md 或结构化提示，交由 AI 生成代码或文档草稿，随后人工运行、核验数值与图件是否与数据一致。问题一提示词原文见 A/问题一/AGENTS.md 及 log_2.md 附录，此处不重复粘贴以免附录过长；支撑材料中保留全文。

C.4 采纳、修改与核验

- 问题一流水线经运行日志确认：IQR 替换 559 格、200 圈样本 40 块、分位阈值 0.993433/0.996103，与正文一致。
- AI 生成的解释性文字（如 NEWSTRUCTURE 含义、机制讨论）均对照数据表核验后写入；涉及因果的表述已降级为初步观察。
- 问题二至四的模型尚未由 AI 或人工给出最终数值，正文相应部分明确标注为框架，不作为已验证结论。

AI 仅作为辅助工具，论文观点、模型选择与最终结论由参赛队负责。